

ComboGraph

Graph-Based Combo System for Unreal Engine 5

Documentation Version 1.0

Author: Prasanna Keerthivasan

Support: prasannak1999@gmail.com

GitHub: <https://github.com/PrasannaK19999/ComboGraphPlugin>

Compatible with: Unreal Engine 5.5 - 5.7

Module: Runtime | Network Replicated: No | Platforms: Win64, Mac, Linux

Overview

ComboGraph is a production-ready combat sequencing plugin for Unreal Engine 5. It provides a graph-based system where each node in a combo chain holds an AnimMontage and a map of GameplayTag-to-next-node transitions. Input resolves in $O(1)$ via TMap lookup -- no linear scans, no state machine bloat regardless of how many combo branches you author.

Combo windows are defined directly on the montage timeline using a custom AnimNotifyState bar. Animators control exactly which frames accept input without touching C++. A fallback decay timer activates automatically when no notify state is placed on the montage.

System Requirements

- Unreal Engine 5.5, 5.6, or 5.7
- Depends on: GameplayTags (first-party Epic plugin, enabled by default in all UE5 projects)
- No third-party (non-Epic) libraries or external dependencies

Installation

From Fab Marketplace (Recommended)

1. Open the Epic Games Launcher.
2. Go to Library and find ComboGraph in your vault.
3. Click Add to Project and select your project, or click Install to Engine to install engine-wide.
4. Open your project in Unreal Engine.
5. Go to Edit -> Plugins -> search for ComboGraph -> enable it -> restart the editor when prompted.

From GitHub (Manual)

1. Download or clone the repository.
2. Copy the ComboGraph folder into your project's Plugins/ directory (create the folder if it does not exist).
3. Open your project -- Unreal Engine will prompt to rebuild the plugin. Click Yes.
4. Go to Edit -> Plugins -> search for ComboGraph -> enable it -> restart the editor.

Quick Start

Step 1 -- Define GameplayTags

Open Project Settings -> GameplayTags and add your input tags. Example:

```
Input.Light
Input.Heavy
Input.Special
```

Step 2 -- Create a ComboGraphDataAsset

1. Right-click in the Content Browser -> Miscellaneous -> Data Asset.
2. Select UComboGraphDataAsset as the class and name it (e.g. DA_Combo_LightChain).
3. Fill in the Root Node fields:
 - DisplayName -- a human-readable label for the node (e.g. Light1)
 - Montage -- assign the AnimMontage for this attack
 - MontageSection -- optional, leave None unless using montage sections for variants
 - DecayTime -- idle window in seconds after the montage ends (fallback when no notify state is placed)
 - Transitions -- add entries where Key = GameplayTag input and Value = next FComboNodeData (nested inline)

Nesting Transitions inline builds the full combo tree inside one DataAsset. A node with an empty Transitions map is a leaf node -- the combo ends and resets at that point.

Figure 1 -- ComboGraphDataAsset: Root Node properties as shown in the Unreal Engine Details panel

DA_Combo_LightChain (UComboGraphDataAsset)	
v Root Node	(FComboNodeData)
Display Name	Light1
Montage	AM_LightAttack
Montage Section	None
Decay Time	0.5
v Transitions	(1 element)
[0] Key	Input.Heavy
Value	FComboNodeData > ...

Step 3 -- Create a DataTable

1. Right-click in the Content Browser -> Miscellaneous -> Data Table.
2. Set the Row Structure to FComboGraphTableRow.
3. Add rows mapping starting input tags to combo chain DataAssets:

Row Name	InputTag	ComboChain
LightCombo	Input.Light	DA_Combo_LightChain
HeavyCombo	Input.Heavy	DA_Combo_HeavyChain

Row Name is the DataTable row key (any unique string). InputTag and ComboChain are the struct properties.

If OnComboInput is called with a tag that has no matching row in this DataTable, the call is silently ignored -- no error or log message is produced. Verify your tags match exactly.

Step 4 -- Add UComboManagerComponent to your Character

In Blueprint: Add Component -> search for ComboManagerComponent -> assign your DataTable in the Details panel.

In C++:

```
// In your character header
UComboManagerComponent* ComboManager;

// In constructor
ComboManager = CreateDefaultSubobject<UComboManagerComponent>(TEXT("ComboManager"));
```

In the Details panel, set the ComboDataTable property to the DataTable you created in Step 3.

Step 5 -- Wire Input

Call OnComboInput(FGameplayTag) from your input bindings. The component handles everything from that point.

Blueprint: drag off your component, call OnComboInput, and pass your gameplay tag.

C++ with Enhanced Input:

```
void AMyCharacter::OnLightAttackPressed()
{
    ComboManager->OnComboInput(
        FGameplayTag::RequestGameplayTag(FName("Input.Light")));
}
```

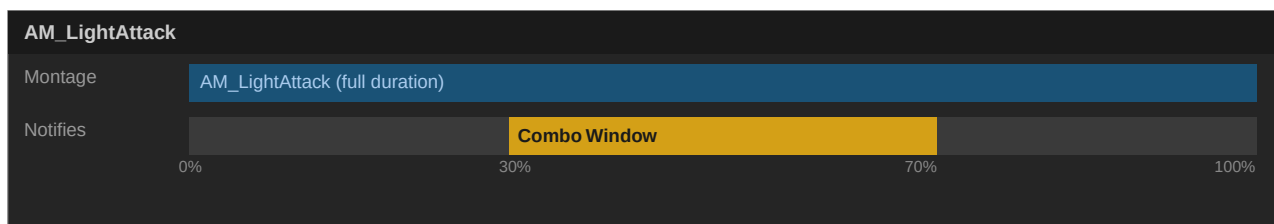
Combo Window -- AnimNotifyState

The Combo Window notify state defines the exact frames on the montage timeline where combo input is accepted. It is the recommended way to control transition timing -- it gives animators direct control without requiring code changes.

How to Place the Notify State

1. Open your attack AnimMontage in the Montage Editor.
2. Right-click on a Notify Track -> Add Notify State -> select Combo Window.
3. Drag the left and right handles to define the input window duration. A typical setup places the window from 30% to 70% of the animation -- covering the moment the hit lands through the recovery phase.

Figure 2 -- AnimMontage timeline: Combo Window notify state bar placed from 30% to 70%



Behavior

- NotifyBegin -- opens the input window. If input was already buffered before the window opened, the transition fires immediately.
- NotifyEnd -- closes the input window. If no input was received, the combo resets to idle.

Fallback Behavior (No Notify State)

If no Combo Window notify state is placed on the montage, the system falls back automatically. After OnMontageEnded fires, the window opens and stays open for DecayTime seconds (defined on the DataAsset node). If no input arrives within that window, the combo resets. This keeps single-attack montages working out of the box.

Runtime Combo Graph Swap

Call SetWeaponDataTable() to swap the full combo graph at runtime. This supports any context -- weapon changes, stance transitions, character state changes:

```
// Blueprint or C++
ComboManager->SetWeaponDataTable(NewDataTable);
```

If a combo is currently active when this is called, it is interrupted cleanly before the new graph is built. The new DataTable is processed immediately and the system is ready for input.

Blueprint API Reference

UComboManagerComponent

Function	Description
OnComboInput(FGameplayTag)	Call from input bindings. Starts a new combo on first press. Routes input to the active combo during a chain.
SetWeaponDataTable(UDDataTable*)	Swaps the active combo graph at runtime. Interrupts any running combo cleanly.
GetActiveGraph()	Returns the active UComboGraph while a combo is running, nullptr when idle. Pass to UComboGraphQueryLibrary functions.
OnComboWindowOpened()	Called automatically by the Combo Window notify state. Do not call manually.
OnComboWindowClosed(UAnimSequenceBase*)	Called automatically by the Combo Window notify state. Do not call manually.

UComboGraph -- Delegates

Bind to these delegates to react to combo state changes. Access the active graph via ComboManager->GetActiveGraph():

Delegate	Fires When
OnComboBegin	The first transition in a new combo chain fires.
OnComboOnGoing	Every subsequent transition after the first.
OnComboEnd	The combo ends -- either decay expired or a dead-end tag was received.

UComboGraphQueryLibrary -- Blueprint Function Library

Static utility functions available anywhere in Blueprint or C++. To get the UComboGraph reference, call ComboManager->GetActiveGraph() on your UComboManagerComponent.

Function	Returns	Description
GetCurrentComboDepth(UComboGraph*)	int32	How many transitions have fired in the current combo chain.
GetComboHistory(UComboGraph*)	TArray<FGameplayTag>	Ordered list of all input tags fired so far in the current combo.
GetLastComboTag(UComboGraph*)	FGameplayTag	The most recent input tag that triggered a transition.
IsComboActive(UComboGraph*)	bool	True if a combo chain is currently running.

IComboWindowListener Interface

UComboManagerComponent implements this interface automatically. If you need to drive the combo window from a custom component, implement IComboWindowListener on that component:

```
virtual void OnComboWindowOpened_Implementation() override;
virtual void OnComboWindowClosed_Implementation(UAnimSequenceBase* Animation) override;
```

The Combo Window AnimNotifyState searches the owning actor for any component implementing this interface and calls it. Only the first matching component is called per notify event.

Data Structures Reference

FComboNodeData (Struct)

Defines a single node in a combo chain. Used inside UComboGraphDataAsset.

Property	Type	Description
DisplayName	FName	Human-readable label. Used for debug display only.
Montage	UAnimMontage*	The animation montage to play when this node activates.
MontageSection	FName	Optional section name within the montage. Leave None to play from start.
DecayTime	float	Fallback idle window in seconds. Used when no Combo Window notify state is placed.
Transitions	TMap<FGameplayTag, FComboNodeData>	Maps input tags to the next FComboNodeData. Self-nesting builds the full combo tree. Empty map = leaf node. Note: stored internally as FInstancedStruct to support recursive nesting -- the editor picker shows FComboNodeData.

FComboGraphTableRow (DataTable Row Struct)

The row type for the DataTable assigned to UComboManagerComponent.

Property	Type	Description
InputTag	FGameplayTag	The input tag that starts this combo chain (e.g. Input.Light).
ComboChain	UComboGraphDataAsset*	The DataAsset holding the full combo tree for this chain.

Important Notes

- Network replication is not supported in v1. The system is local only.
- Mac and Linux compilation is supported -- the plugin has no platform-specific code. Windows is the verified and tested platform.
- The plugin contains no content (no Blueprint assets, no maps, no textures). It is a pure C++ runtime module.
- Multiple combo chains are supported simultaneously via separate DataTable rows. Each starting input tag maps to its own independent graph.
- The system is stateless between combos. After OnComboEnd fires, the component is immediately ready for the next input.

Troubleshooting

Combo does not start on input

- Verify ComboDataTable is assigned on the component in the Details panel.
- Confirm the InputTag in the DataTable row exactly matches the tag passed to OnComboInput. A mismatch causes a silent no-op -- no error is thrown.
- Check that the GameplayTag is registered in Project Settings -> GameplayTags.

Combo gets stuck and does not reset

- Ensure the AnimMontage assigned to the node has a valid length and plays correctly standalone.
- If using the fallback (no notify state), verify DecayTime is greater than 0.
- Place the Combo Window AnimNotifyState on the montage timeline for precise control.

Transitions fire twice

- This is guarded internally. If it occurs, verify you are not calling OnComboInput from multiple input bindings for the same button press.